

Optimal DC Motor Control with a Switch-Mode Power Supply – Simulation

Numerical Optimal Control in Science
and Engineering

Project Report (WS 2025-2026)

Author: Antti Paasi

Supervisor: Andrea Ghezzi

Abstract— Optimal control schemes can potentially improve energy efficiency and extend the lifespan of power electronic converters. In this project, an approach for optimizing switch-mode power supply usage is tested. A DC motor model is controlled with a Buck converter, while preventing aggressive switching frequency changes. Three optimal control schemes are simulated: Linear Quadratic Regulator, Open Loop Optimal Control and Model Predictive Control. All scripts are implemented in MATLAB. CasADi and Control System Toolbox are used.

I. INTRODUCTION

In this project, a DC motor controlled with a Buck converter is modelled, and three different optimal control methods are simulated in MATLAB. The shaft speed of the DC motor is forced to follow a reference signal while preventing aggressive control changes. The motivation for implementing optimal control is the hypothesis that restricting switching speed changes could extend converter lifespan and improve energy efficiency.

After deriving a discrete-time model, LQR control is simulated with a constant reference speed. Then, a more complex reference is used, and open loop optimal control is simulated. Lastly, MPC with a Kalman filter is simulated as an enhanced and applicable version of the second scheme. All MATLAB scripts are available in the GitHub repository [1].

II. DC MOTOR AND BUCK CONVERTER SYSTEM MODEL

A continuous-time state-space model for a DC-motor is derived in [2]. The model is given by equations

$$\dot{x} = \begin{bmatrix} -\frac{b}{J} & -\frac{K}{J} \\ \frac{K}{L} & -\frac{R}{L} \end{bmatrix} x + \begin{bmatrix} 0 \\ 1/L \end{bmatrix} u, \quad (1.1)$$

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} x, \quad (1.2)$$

where $x = [\dot{\theta} \ i]^T$ is the state vector containing the motor shaft angular speed $\dot{\theta}$ and armature current i . The control signal $u = V_I \cdot D$, where V_I is the converter input voltage and D is the duty cycle, which is explained subsequently. Other parameters are shaft friction coefficient b , shaft moment of inertia J , electromotive force constant K , motor inductance L , and motor resistance R .

We use a simplified model for the Buck converter, because we want to use duty cycle as the control signal. According to [3, p. 6], the continuous steady-state model for Buck converter output voltage is

$$V_O = (V_I - V_{DS}) \times D - V_D \times (1 - D) I_L \times R_L, \quad (2)$$

where V_I and D are again the converter input voltage and duty cycle, V_{DS} , V_D and $I_L \times R_L$ are voltage drops in the circuit. However, we simplify further by assuming an ideal circuit, leading to

$$V_O = V_I D. \quad (3)$$

Parameters values are not chosen from an actual motor datasheet. We copy values for J and b from [2]. Parameters R and L are calculated with the second order damping model

$$G(s) = \frac{K \omega_n^2}{s^2 + 2 \zeta \omega_n s + \omega_n^2}, \quad (4)$$

where K is the DC gain, ω_n is the natural frequency and ζ is the damping coefficient. $G(s)$ is a function in the frequency domain. Using the damping model is useful, because a relatively fast-enough step response with no overshoot is needed for a realistic DC motor model. With the damping model, we can directly work with parameters that determine how the system oscillates. In this case, we design system behavior with resistance and inductance values, which could in principle be decided with component choices in real applications. First, the DC motor transfer function from [2] is derived to the second order damping model form. Then, the damping coefficient is set to $\zeta=1$ (critically damped). Values for R and L , as well as the resulting natural frequency ω_n are then solved. A step-by-step solution is available in the GitHub repository [1]. The goal of this project is to practice optimal control, not precise motor modeling, so this approach is acceptable. Parameter values used in the project are represented in table 1.

TABLE I. SYSTEM PARAMETER VALUES

Parameter	Description	Unit	Value
J	Shaft Moment of Inertia	kgm^2	0.01
b	Shaft Friction Coefficient	Nms	0.1
K	Electromotive Force Constant	Vs	0.7
R	Motor Resistance	Ω	5.1
L	Motor Inductance	H	0.09
V_i	Converter Input Voltage	V	24
T_s	Sampling Time	s	0.01

As shown in table 1, sampling time $T_s = 0.01$ is used. We use the forward-Euler discretization method presented in [4, p. 50], because our sample time is sufficiently short. Finally, our discrete-time system model is

$$x(k+1) = \begin{bmatrix} 0.9000 & 0.7000 \\ -0.0778 & 0.4333 \end{bmatrix} \begin{bmatrix} \dot{\theta}(k) \\ i(k) \end{bmatrix} + \begin{bmatrix} 0 \\ 0.1111 \end{bmatrix} \cdot 24 \cdot D(k) \quad (5.1)$$

$$y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}(k) \\ i(k) \end{bmatrix}. \quad (5.2)$$

A step input is displayed in figure 1 and system response to that input is shown in figure 2.

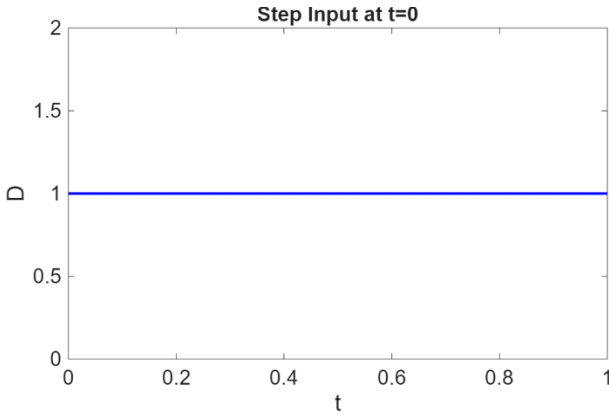


Fig. 1. Step Input

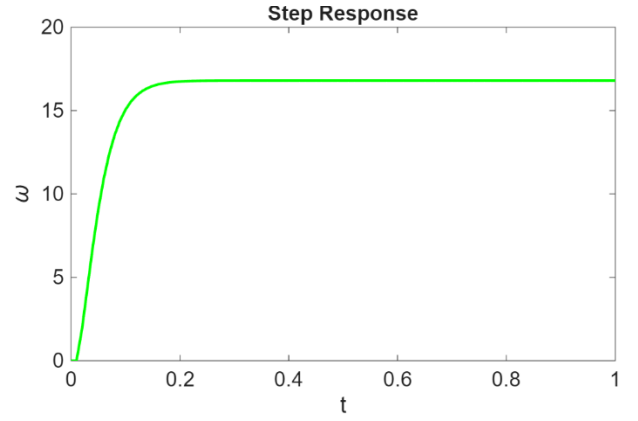


Fig. 2. Step Response

The discrete-time system has a double pole at 0.6667 in the Z-domain, which lies inside the complex unit circle. Therefore, the system is stable.

III. LINEAR QUADRATIC REGULATOR

Theory regarding the LQR implementation was learned from [5, p.126] and the feedback loop idea from [6]. A basic principle called the Bryson's rule is used to tune the state weight and control weight matrices. This rule is stated in the LQR Design documentation by Mathworks [7]. Bryson's rule says that the matrices should be tuned according to

$$Q_{i,i} = \frac{1}{(\text{maximum state error})^2}, \quad i \in \{1, 2, \dots, n\}. \quad (6)$$

Values $Q_{i,i}$ are set to the diagonal, other values are zero. In this project, applying the rule leads to the matrices

$$Q = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} \\ R = [100].$$

The matrix Q is the state weight matrix and R is the control weight matrix, here a scalar. Because our goal is to minimize converter switching changes, we define using controls as being more costly than state errors. In the simulation script, the optimal feedback control matrix K is calculated with MATLAB function `dlqr`.

The idea is to linearize the system around a constant set point. The simulation is run for a 0.3s timeframe. Three different situations with different initial shaft speed values ω_0 and goal shaft speeds ω_g are simulated. Uniformly distributed noise is added to the state vector in each timestep to mimic real world behavior. The noise is defined with MATLAB command

$$\text{randi}([-1000, 1000]) * 0.00005 * \text{ones}(2, 1),$$

which leads to deviations from calculated system state in the range of $10e-3$. This is a fairly small error, which is handled well by the feedback loop. A higher level of noise would demand a filter. Both using higher noise levels and filtering noisy measurements are tested in the next chapters.

Due to the control signal being limited between values 0 and 1, the motor saturation speed is around 17 1/s. Unfeasible controls are clipped in the simulation. Results are presented in figures 3 and 4. In figure 4, initial shaft speeds ω_0 are the vertical axis values at time 0 and the dotted lines are the set points.

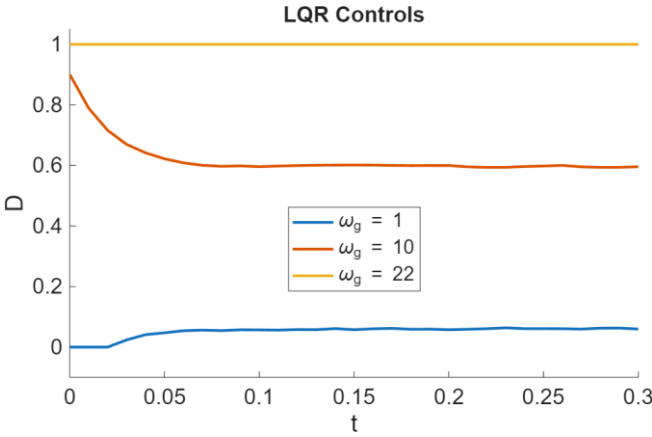


Fig. 3. LQR controls for different initial values and set points. Time on horizontal axis, duty cycle on vertical axis.

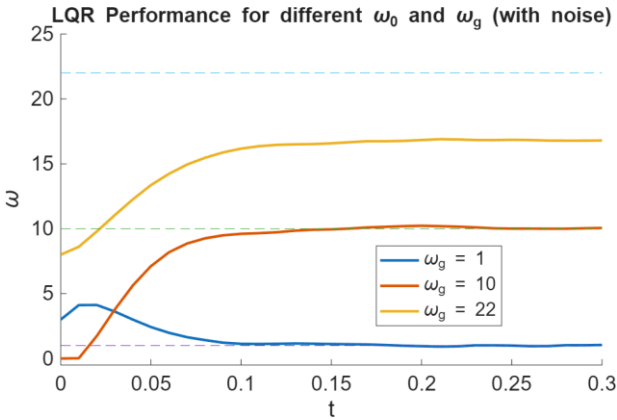


Fig. 4. LQR performance for different set points and initial values. Dotted lines are the references.

Here, the LQR scheme results in a relatively fast regulation. As said, the reference speed $\omega_g = 22$ 1/s is unfeasible, so it leads to saturation. This kind of regulation can be useful for applications with set points that don't change rapidly. It is worth noting that a well-tuned PI controller would probably perform equally well here.

IV. OPEN LOOP OPTIMAL CONTROL

Now we want to dive deeper into optimization and solve an open loop optimal control problem with CasADi and its **qpso1** solver. The solver finds an optimal way of controlling the system for one timeframe so that the motor shaft speed closely follows a more complicated reference signal. The objective function to be solved is

$$\min \sum_{k=0}^{N-1} [\omega_e(k)^T \cdot q \cdot \omega_e(k) + D(k)^T \cdot r \cdot D(k)] \quad (7.1)$$

$$+ \omega_e(N)^T \cdot q \cdot \omega_e(N)$$

s.t.

$$x(0) = x_0, \quad (7.2)$$

$$d(0) = d_0, \quad (7.3)$$

$$x(k+1) = A_d \cdot x(k) + B_d V_I D(k), \quad (7.4)$$

$$0 \leq D(k) \leq 1, \quad (7.5)$$

$$-dd_{max} \leq D(k) - D(k-1) \leq dd_{max} \quad (7.6)$$

$$\omega(N) - ref(N) = 0. \quad (7.7)$$

Parameters and variables are explained below.

$\omega_e(k) = ref(k) - \omega(k)$ is shaft speed error compared to reference speed signal,

$x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ is the initial state,

$d_0 = 0$ is the initial control,

$D(k)$ is the duty cycle,

$dd_{max} = 0.07$ is the maximum control change in one timestep (in code, marked as **max_dd**),

$q = [1000]$ is the state weighting matrix (here, a scalar) and $r = [100]$ is the control weighting matrix (here, a scalar).

Sequential approach is used for implementation, so only $D(k), k = 1, 2, 3, \dots, N-1$ are decision variables. The matrices are tuned using the same principle as in chapter III, equation (5). Emphasis is again more on the control cost. In addition, we add constraints for maximum change of control per timestep. We mark this as **max_dd** = 0.07. Because the control signal (converter duty cycle) is between 0 and 1, the physically feasible maximum change would be 1. This parameter choice greatly prevents aggressive control. In figure 5, the system response to optimal controls is displayed against the reference signal.

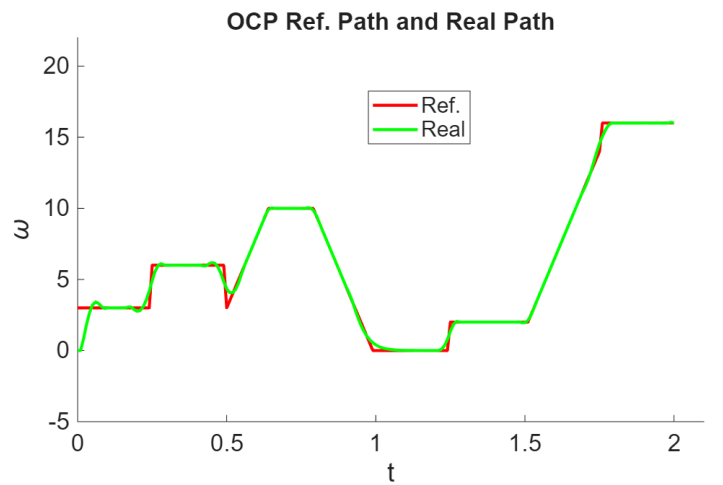


Fig. 5. Open Loop Control Performance

Open loop control performs well when the system behavior is deterministic, as is the case here. If deviations from

the system model would occur, e.g. noise would be present, open loop control would start working poorly. A problem like this can be solved with feedback control and noise filtering. This approach is put to test in the next chapter.

V. MODEL PREDICTIVE CONTROL WITH KALMAN FILTER

Theory regarding MPC was learned from [6, p.261-266] and [7]. A more practical example was studied with documentation and scripts from [8]. In MPC simulation, the objective function and constraints are now

$$\min \sum_{k=0}^{N-1} [\omega_e(k)^T \cdot q \cdot \omega_e(k) + D(k)^T \cdot r \cdot D(k)] \quad (8.1)$$

$$+ \omega_e(N)^T \cdot q \cdot \omega_e(N) \quad (8.2)$$

$$\text{s.t.} \quad (8.2)$$

$$x(0) = x_0 \quad (8.3)$$

$$D(0) = d_0 \quad (8.4)$$

$$x(k+1) = A_d \cdot x(k) + V_l D(k), \quad (8.5)$$

$$0 \leq D(k) \leq 1 \quad (8.6)$$

$$-dd_{\max} \leq D(k) - D(k-1) \leq dd_{\max}, \quad (8.7)$$

Terminal constraint is not used, because in this case, it causes the solver to fail. Fortunately, the solver works even without a terminal constraint. The difference to the open loop problem is, we now solve many optimization problems in a loop. In this simulation, `control_window = 5` is used, which means that we only calculate controls for 5 time points (0.05s forward) in each iteration. Only the first control is implemented after each solution. In each iteration, the solver takes the latest state and the latest control as initial parameters. The latter is important, because of the `max_dd` restriction.

Again, noise is used to disturb the system. The noise is uniformly distributed and it is added to both the calculated next state and the current state (adding noise both to the process and the measurement). In general, if the noise level is low, MPC and open loop perform similarly well. However, when noise level grows, both start to deviate from the desired path. With open loop, the explanation is simple: no feedback leads to no knowledge of the occurring deviation, and thus controls are not changed. For MPC, the controller thinks that the noisy measurements are the actual system states. This leads to the controller aggressively trying to get back on the desired path, which does not work too well either. In this project, the goal is to change the control signal (switching frequency) as little as possible and to follow the reference path. As a solution, a Kalman filter is utilized to filter process and measurement noise, so that the controller gets better estimates of the actual state of the system.

Kalman filter is an algorithm for estimating system parameters. It is commonly used in applications where accurate measurements are not available. It uses covariance matrices P_k (state error), R_k (measurement noise) and Q_k (process noise) to estimate the actual system state. First, the matrices are initialized. By trial and error, the following initial values for the matrices were chosen for this project.

$$P_k = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix},$$

$$R_k = [10] \text{ (scalar) and}$$

$$Q_k = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}.$$

In addition, the system state estimate is initialized as

$\hat{x} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. In the algorithm loop, the noisy system state is observed, the state estimate is calculated and the matrix P_k is updated. In this simulation, we calculate process noise $\mathbf{w}$ and measurement noise $\mathbf{v}$ with MATLAB commands

$$\mathbf{w} = \text{randi}([-1000, 1000]) * 1e-3 * \text{ones}(2, 1)$$

and

$$\mathbf{v} = \text{randi}([-1000, 1000]) * 1e-3.$$

The Kalman filter equations are presented below. In simulation, they are carried out in a loop.

$$x_p(k+1) = A_d \hat{x}(k) + B_d \cdot V_l \cdot D(k) \quad (9.1)$$

$$x_m(k+1) = x_p(k+1) + \mathbf{w} \quad (9.2)$$

$$y_m = C_d \cdot x_m + \mathbf{v} \quad (9.3)$$

$$P_k = A_d \cdot P_k \cdot A_d^T + Q_k \quad (9.4)$$

$$K_k = \frac{P_k \cdot C_d^T}{C_d \cdot P_k \cdot C_d^T + R_k} \quad (9.5)$$

$$\hat{x}(k+1) = x_p(k+1) + K_k \cdot (y_m - C_d \cdot x_p(k+1)) \quad (9.6)$$

$$P_k = P_k - K_k \cdot C_d \cdot P_k, \quad (9.7)$$

where A_d , B_d and C_d are discretized system matrices presented in equation 5, x_p is the state prediction based on last state estimate and current control, x_m is the simulated noisy state, y_m is the simulated noisy measurement based on noisy state, and K_k is the Kalman gain used to calculate the state estimate. Note that P_k is updated twice, first with process noise knowledge and then with the calculated Kalman gain.

Kalman filter assumes normally distributed noise, while here the noise is uniformly distributed. The resulting state estimate is still useful. It also reflects a realistic situation where the exact noise distribution is not known. A block diagram of the MPC and Kalman filter system is shown in figure 6. Simulation results without the filter are shown in figure 7, and simulation results with the filter are shown subsequently in figure 8. In figure 7, term “measured path” is used, while in figure 8 we can confidently say that the shown path is the real system speed.

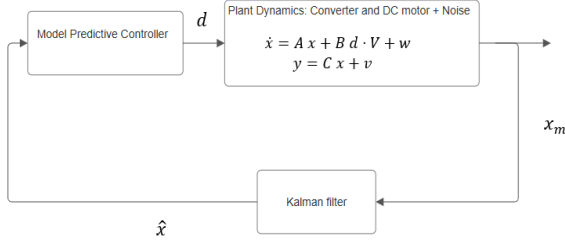


Fig. 6. Block Diagram for MPC + Kalman filter control

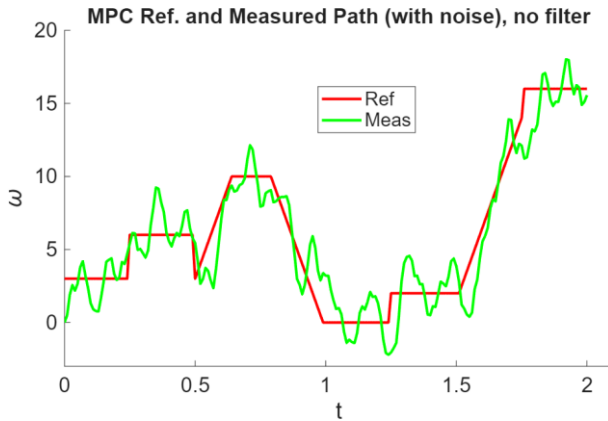


Fig. 7. MPC Performance without Kalman filter

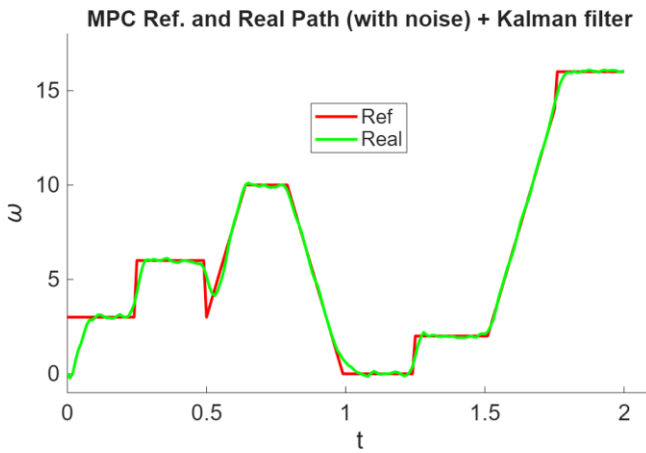


Fig. 8. MPC performance with Kalman filter

When Kalman filter is used, MPC performs well in a noisy environment. Even though the noise causes reasonably big deviations from the system model based calculations, the MPC controller is still able to control the motor in the desired way. Noise is often a problem in real applications, therefore a well-tuned filter is a necessity. It is also worth adding that this type of filtering is only usable with feedback control, so we could not utilize a Kalman filter in the open loop solution.

The simulation time is also an interesting result. Simulating the 2s timeframe with MPC and Kalman filter only takes about 0.2s to complete, so in principle the solver is fast

enough for real world applications. This topic is briefly discussed in the next chapter.

VI. SUMMARY

In this project, a model for a DC Motor and converter system was derived and three different optimal control methods were simulated. The goal was to force the motor to follow a reference speed signal while preventing aggressive control changes, because of the converter lifespan lengthening and energy saving goals. In terms of performance, we saw that LQR is a suitable solution for relatively simple reference signals. MPC with Kalman filter is superior to open loop optimal control when noise is present, as is often the case in the real world. A natural next step would be to deploy these schemes in real-time control in hardware and test if they really extend the lifespan and improve energy efficiency of a power electronic converter. The controller algorithm could be, for example, first converted to embeddable C code with the acados interface.

REFERENCES

- [1] GitHub repository, Optimal-DC-Motor-Control-Simulation, <https://github.com/anttipaasi/Optimal-DC-Motor-Control-Simulation>
- [2] DC Motor Speed: System Modeling, University of Michigan, <https://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed§ion=SystemModeling>
- [3] Understanding Buck Power Stages in Switchmode Power Supplies, Application Report by Texas Instruments, 1999, <https://www.ti.com/lit/an/slva057/slva057.pdf>
- [4] Model predictive control of high power converters and industrial drives, Geyer, Tobias, 2017
- [5] Numerical Optimal Control (Draft), Diehl, Moritz, Gros, Sébastien, 2024. Available online at <https://www.syscop.de/files/2024ws/NOC/book-NOCSE.pdf>
- [6] Numerical Optimal Control in Science and Engineering, exercise sheet 7, available at the course website <https://www.syscop.de/teaching/ss2020/numerical-optimal-control-online>
- [7] Linear-Quadratic Regulator (LQR) design, Mathworks, Look at Q - state cost weight matrix definition), <https://se.mathworks.com/help/control/ref/lti.lqr.html>
- [8] Model Predictive Control – Theory, Computation and Design, Rawlings, James B., Mayne, David Q., Diehl, Moritz M., 2024
- [9] OCP Playground, GitHub repository by Florian Messerer, <https://github.com/fmesserer/ocp-playground>
- [10] Kalman filter explained simply, <https://thekalmanfilter.com/kalman-filter-explained-simply/>